ADARE: Learning-Guided Adaptive Control for Many-Objective Workflow Scheduling in Heterogeneous Edge/Fog/Cloud Systems

Roland Cedric Tayo and Sonia Yassa

CY Cergy Paris University, France
rolandcedric.tayo@yahoo.fr, sonia.yassa@cyu.fr

Abstract. ADARE (Adaptive Data-driven Algorithm for Resource Evolution) is a learning-guided many-objective workflow scheduler for heterogeneous Edge/Fog/Cloud systems. The revised algorithm is no longer positioned as a parameter-tuned extension of NSGA-III, but as an autonomous search-control architecture: a Pareto-contextual controller decides how variation should behave while a reference-vector environmental selector maintains many-objective pressure. The controller discretizes the search state by generation phase, population diversity, and stagnation, and each context maintains its own UCB estimates over crossover operators. Rewards are computed from dominance events, objective-wise normalized improvement, balanced multi-objective progress, and genotype novelty, rather than from a fixed log-scalar proxy. ADARE couples this controller with density-aware mutation, bounded archive guidance, and exported controller traces for interpretability. We evaluate ADARE on Montage_25, CyberShake_30, and Epigenomics_24 with a strict fairness protocol: same evaluator, same budget, same seeds, and paired 20-run comparisons per benchmark. Across the final protocol, ADARE wins 18/18 quality comparisons and 15/15 core-quality comparisons (Hypervolume (HV), Inverted Generational Distance (IGD), spacing, epsilon, coverage). Mean gains are +26.80% on core quality, +10.70% on best-objective metrics, and +5.50% on runtime versus NSGA-III. We also add 20-repeat 1000-task and WfCommons-generated 3000-task stress tests against NSGA-III, QL-NSGA-III, OVEA-style, and QMOEA/D-AWA-style baselines, plus a broader screening suite with NSGA-II and MOEA/D. Across the confirmatory large-scale protocol, ADARE wins 72/80 core-metric comparisons at 1000 tasks and 86/100 at 3000 tasks; mean core-quality gains are +18.08% and +19.26% versus NSGA-III, +7.26% and +3.54% versus QL-NSGA-III, +49.74% and +21.39% versus OVEA-style, and +29.44% and +17.92% versus QMOEA/D-AWA-style. The contribution is therefore a workflow-specific adaptive control algorithm for heterogeneous scheduling, not merely a recombination of a static MOEA backbone and a generic bandit.

Keywords: Online learning · Many-objective optimization · NSGA-III · Multi-armed bandits · Edge/Fog/Cloud · Workflow scheduling

1 Introduction

Modern distributed applications increasingly execute over heterogeneous computing strata, where edge nodes, fog servers, and cloud resources expose markedly different latency, bandwidth, energy, and cost profiles. In this context, workflow scheduling is not only a resource allocation problem; it is also a *control problem under uncertainty* over a search landscape that changes as candidate schedules evolve. We therefore study ADARE (Adaptive Data-driven Algorithm for Resource Evolution) as a many-objective adaptive scheduler whose core contribution is online control of variation, diversity recovery, and archive guidance under heterogeneous workflow feedback.

NSGA-III remains a strong baseline for many-objective optimization because of its reference-point selection mechanism [7]. However, workflow-scheduling implementations commonly keep crossover and mutation probabilities fixed across the entire run. In heterogeneous Edge/Fog/Cloud settings, this assumption is fragile: an operator that is useful for early exploration can later damage partial structures once the population starts refining narrow trade-off regions. In practice, this can reduce either convergence pressure or diversity exactly when the search distribution starts to shift.

In our early paired runs, this phase-dependent behavior appeared repeatedly in the convergence traces before it became obvious in the summary tables. That pushed us away from treating ADARE as a tuned NSGA-III derivative and toward a cleaner algorithmic identity: ADARE is a workflow-specific search controller that can currently use reference-vector selection as its environmental survival mechanism. This framing follows a learning-driven decision view: the optimization engine acts as the environment, operator choice is the action, and search feedback becomes the training signal.

Contributions. The four contributions below came out of repeated implementation iterations in our paired runs and reruns.

1. First, we introduce a *Pareto-contextual operator controller*: UCB estimates are conditioned on generation phase, diversity regime, and stagnation state, so ADARE learns different crossover preferences for exploration, recovery, and refinement.
2. Second, we replace the former log-scalar reward with a bounded multi-signal reward that combines Pareto dominance, objective-wise normalized improvement, balanced progress across objectives, and genotype novelty.
3. Third, we add density-aware mutation, archive guidance, and controller traces, while keeping the environmental selection mechanism fixed for fair attribution in paired experiments.
4. Finally, we report a statistically grounded evaluation on three workflow instances with **20 paired runs** per benchmark, together with computational complexity analysis and runtime profiling.

The remainder of this paper is structured as follows: Section 2 reviews related work, Section 3 presents the problem modeling, Section 4 describes the proposed approach, and Section 5 details the experimental protocol and experimental results. Section 6 discusses the findings and their limitations, and Section 7 concludes the paper and outlines future research directions.

2 Related Work

2.1 Workflow Scheduling in Edge/Fog/Cloud Systems

Workflow scheduling in heterogeneous distributed systems is classically formulated as a Directed Acyclic Graph (DAG) mapping problem with precedence and communication constraints. Heuristic methods such as Heterogeneous Earliest Finish Time (HEFT) remain competitive for makespan-focused scheduling because of their low computational cost [1], yet they are not naturally designed for multi-objective trade-offs involving latency, energy, and monetary cost. The fog computing paradigm further amplifies this challenge by introducing an intermediate layer with distinct communication and compute properties [3].

For this reason, many-objective formulations are more appropriate in heterogeneous edge, fog, and cloud orchestration than single-score scheduling. The scheduler should expose trade-off solutions rather than a single operating point. In the workflow community, benchmark instances such as Montage, CyberShake, and Epigenomics are widely used to stress both compute and communication behavior; we rely on public workflow repositories and Workflow Commons (WfCommons)-style instances for reproducible experimentation [6].

2.2 Baseline Many-Objective Evolutionary Algorithms

With a moderate objective count, many studies still start from Non-dominated Sorting Genetic Algorithm II (NSGA-II) [4]. In our comparisons, NSGA-III keeps the same lineage and uses reference points to retain selection pressure in many-objective settings [7]. Multiobjective Evolutionary Algorithm based on Decomposition (MOEA/D) [5] is different. It decomposes the problem into coordinated scalar subproblems. We keep this trio because it exposes distinct search biases during runs: dominance ranking, reference-point niching, and decomposition.

In practice, however, workflow scheduling studies often keep crossover and mutation schedules fixed because dynamic operator policies are harder to instrument and compare reproducibly. That engineering convenience becomes a methodological limitation in our setting. ADARE keeps the many-objective selection backbone intact (NSGA-III in the current artifact) and changes the control policy around operator choice and mutation behavior as the search state evolves.

2.3 Learning-Guided Adaptive MOEAs

Adaptive operator selection and reinforcement-learning-assisted evolutionary algorithms have become increasingly relevant in recent years. OVEA introduces Q-learning-driven operator adaptation in a reference-vector-based Multiobjective

Evolutionary Algorithm (MOEA) [12]. A recent survey by Song et al. synthesizes the broader design space of reinforcement learning for evolutionary algorithms and highlights online control as a key direction for scalable adaptive search [13]. More recently, Wang et al. propose an MRL-MOEA framework based on multi-role learning in a many-objective setting [14], while QMOEA/D-AWA uses Q-learning inside an adaptive-weight MOEA/D framework for low-carbon flexible job-shop scheduling [15].

Bandit-based adaptive operator selection itself is not new; dynamic MAB mechanisms were studied for evolutionary algorithms before the present work [9,10]. We therefore do not claim novelty from the use of UCB alone. The contribution is the *Pareto-contextual* specialization for heterogeneous workflow scheduling: the controller conditions its estimates on search phase, diversity, and stagnation, and its reward uses normalized multi-objective evidence rather than a single scalar proxy. These studies support the broader premise of *learning-guided evolutionary control*. In our setting, the key distinction is not only whether learning is used, but where it is inserted in the optimization loop, what feedback signal is exposed to the learner, and what runtime cost it introduces. Most published results target generic benchmark suites rather than workflow scheduling with heterogeneous costs and bandwidth asymmetry. ADARE addresses that systems-specific gap while keeping a controller that can be profiled directly on a local workstation.

2.4 Comparative Landscape of Cited Baselines

Table 1 summarizes the roles of the most relevant baselines and adaptive MOEA families discussed above. The table is intentionally split between *algorithmic positioning* and *workflow-comparable artifact availability*: when an original method targets generic continuous benchmarks or manufacturing job-shop instances, we implement an executable workflow adaptation rather than importing non-comparable paper results.

With that positioning in place, we can state the scheduling model and the objective vector actually used by the online control loop.

3 Problem Modeling

3.1 System Model

We consider a workflow represented as a directed acyclic graph (DAG) $G = (V, E)$, where each task $v_i \in V$ carries a computational load and a data transfer footprint, and each directed edge encodes a precedence relation with communication dependency. The execution platform is a heterogeneous resource set R spanning edge, fog, and cloud nodes, each characterized by processing rate, communication bandwidth, monetary processing cost, and power usage.

A candidate schedule is encoded as a task-to-node assignment vector

$$x = (x_1, \ldots, x_{|V|}), \qquad x_i \in \{0, 1, \ldots, |R| - 1\}. \tag{1}$$

Table 1. Compact Comparative Landscape (Baselines and Adaptive MOEA Positioning)

Method / family	Many-obj.	Online learning	Adaptivity	Relevance to this study
NSGA-II [4]	Med.	No	No	Standard baseline for multi-objective optimization; included for methodological positioning.
NSGA-III [7]	High	No	No	Direct paired baseline in the artifact; strong selection, though operator schedule remains static.
MOEA/D [5]	High	No	No	Strong decomposition baseline; included for comparative positioning.
OVEA-style [12]	High	Yes (Q-learning)	Yes	Executable workflow adaptation of reference-vector operator learning.
QMOEA/D-AWA-style [15]	High	Yes (Q-learning)	Yes	Executable workflow adaptation of adaptive-weight decomposition control.
ADARE (this work)	High	Yes (contextual UCB)	Yes	Workflow-specific Pareto-contextual control with fairness-controlled evaluation.

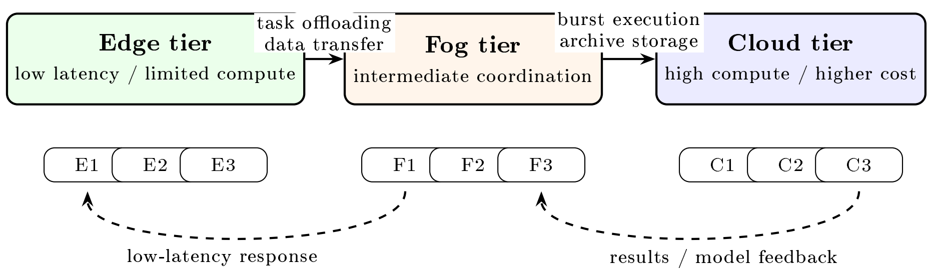

Fig. 1. Simplified Edge/Fog/Cloud execution architecture used to motivate the scheduling model. The heterogeneous compute, bandwidth, and cost properties across tiers create the many-objective trade-offs optimized by ADARE and NSGA-III.

Given a topological order of the DAG, the evaluator simulates task start and finish times under node availability and inter-node transfer delays.

Figure 1 clarifies the systems setting: edge nodes favor low-latency execution, fog nodes provide coordination and intermediate capacity, and cloud nodes provide stronger compute at a different cost/bandwidth profile. This heterogeneity is the reason a static operator schedule can become suboptimal during search.

3.2 Objective Vector

ADARE and NSGA-III optimize the same four-objective minimization vector:

$$\min\ \mathbf{f}(x) = \left(f_{\mathrm{mk}}(x), f_{\mathrm{lat}}(x), f_{\mathrm{cost}}(x), f_{\mathrm{eng}}(x)\right), \tag{2}$$

where:

- f_{mk} is the workflow makespan (maximum task completion time);
- f_{lat} aggregates task completion delays relative to readiness times;
- f_{cost} accumulates compute-time-weighted processing cost across assigned nodes;
- f_{eng} accumulates compute-time-weighted working power as an execution energy proxy.

Let C_i, R_i, I_i, ρ_{x_i}, c_{x_i}, and $p_{x_i}^{\mathrm{wrk}}$ denote completion time, readiness time, task instructions, node processing rate, node cost, and node working power; cost and energy use execution-time weighting I_i/ρ_{x_i}. The evaluator computes these metrics as:

$$\begin{aligned} f_{\mathrm{mk}}(x) &= \max_{i \in V} C_i, \\ f_{\mathrm{lat}}(x) &= \sum_{i \in V} \left(C_i - R_i\right), \\ f_{\mathrm{cost}}(x) &= \sum_{i \in V} \left(\frac{I_i}{\rho_{x_i}}\right) c_{x_i}, \\ f_{\mathrm{eng}}(x) &= \sum_{i \in V} \left(\frac{I_i}{\rho_{x_i}}\right) p_{x_i}^{\mathrm{wrk}}. \end{aligned} \tag{3}$$

This reflects the core edge/fog/cloud tension: faster assignments can reduce completion time while increasing cost or energy, so we target a diverse Pareto set rather than a single operating point.

3.3 Evaluation Targets and Quality Indicators

In addition to objective values, we evaluate search quality using hypervolume (HV), inverted generational distance (IGD), spacing, additive epsilon, and coverage. No single indicator was reliable enough on its own in our runs, so we keep this set to read front quality from complementary angles (dominance coverage, convergence to a reference front, and spread regularity). For ADARE, that distinction matters: the controller is meant to improve search behavior over time, not just one objective extreme.

With the objective vector and evaluation targets fixed, we now describe how the search loop uses this feedback online.

4 ADARE Methodology: Learning-Guided Adaptive Control

4.1 Design Overview

ADARE is formulated as a many-objective adaptive scheduler with three explicit control modules: Pareto-contextual operator selection, density-aware mutation and repair, and bounded archive guidance. The current artifact uses NSGA-III-style reference-vector survival as the environmental selection component because

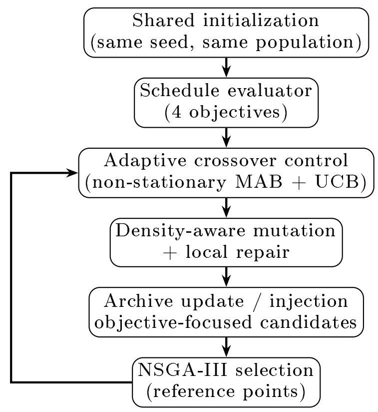

Fig. 2. ADARE as a learning-guided workflow scheduler: online search control surrounds a reference-vector environmental selection step.

it is a strong and interpretable many-objective survival rule; the ADARE contribution is the online control policy surrounding that survival step. Figure 2 summarizes this organization.

We kept this architecture on purpose: a monolithic learner would blur attribution in paired comparisons. ADARE instead makes the controllable parts of the evolutionary loop adaptive while keeping environmental selection explicit, which preserves interpretability and a practical runtime profile.

4.2 Learning Formalization: Pareto-Contextual MAB for Operator Control

At each generation, ADARE selects a crossover operator from a small portfolio (one-point, two-point, and uniform variants). The earlier implementation used one global UCB table. The revised implementation uses a compact contextual bandit because the same operator does not have the same value during early exploration, low-diversity recovery, and late-stage refinement. This is the scheduling-specific part of the learning formulation: the controller state is derived from search dynamics that are visible in heterogeneous DAG scheduling, not from an offline task embedding.

Let $s_t = (\rho_t, d_t, z_t)$ be the controller context at decision step t, where ρ_t is a three-bin generation phase, d_t is a two-bin diversity regime, and z_t is a two-bin stagnation indicator. ADARE therefore maintains 12 small context tables. Let $Q_{s,k}$ be the current quality estimate for operator k in context s, and $N_{s,k}$ its usage count. ADARE chooses:

$$a_t = \arg\max_k \left(Q_{s_t,k} + \sqrt{\frac{2\ln(N_{s_t}+1)}{N_{s_t,k}+1}} \right). \tag{4}$$

The second term maintains exploration inside the current search regime, while the first term exploits operators with strong recent rewards in that same regime.

Reward definition. The reward is computed on the parent-offspring comparison but avoids a single fixed scalarization. Let $P = \{p_1, p_2\}$ and $C = \{c_1, c_2\}$ be the parent and child objective vectors. For minimization, ADARE computes:

$$r_t = 0.45D(P, C) + 0.35I(P, C) + 0.15B(P, C) + 0.05V(P, C), \tag{5}$$

where D rewards child-parent Pareto dominance and penalizes dominated children, I is the mean objective-wise improvement normalized by current population ranges, B is the fraction of objectives improved, and V is genotype novelty measured by normalized Hamming distance to the parents. The reward is clipped to $[-1, 1]$. This keeps the signal cheap enough for per-mating updates while addressing the scale-skew issue of a log-scalar proxy. ADARE updates the selected context-operator value by exponential smoothing:

$$Q_{s_t,k} \leftarrow (1 - \alpha)Q_{s_t,k} + \alpha r_t. \tag{6}$$

This keeps operator control in an online-learning regime while preserving direct interpretability: the implementation exports context labels, operator names, usage counts, and rewards for heatmap-style inspection. Algorithm 1 gives the exact decision/update sequence used per mating event.

Algorithm 1 ADARE Pareto-Contextual UCB Crossover Control

Require: Parent pair (p_1, p_2), population diversity d, stagnation count z, phase ρ
1: Build context $s \leftarrow (\rho, d, z)$
2: Select operator index k with contextual UCB in Eq. (4)
3: Apply crossover operator k to obtain (c_1, c_2)
4: Compute reward r_t with Eq. (5)
5: Update $Q_{s,k} \leftarrow (1 - \alpha)Q_{s,k} + \alpha r_t$ and $N_{s,k} \leftarrow N_{s,k} + 1$
6: Export (s, k, r_t, d, z) to the controller trace
7: **return** (c_1, c_2)

4.3 Density-Aware Mutation and Local Repair

ADARE's mutation layer adapts perturbation effort using generation progress and population diversity. When diversity collapses, mutation pressure increases; when diversity is healthy, mutation stays conservative. A heuristic task-node ranking (processing speed, bandwidth, cost, power) biases part of the mutations toward plausible assignments, with a random fallback to avoid early overcommitment. Algorithm 2 summarizes the mutation and repair sequence used in each generation.

Algorithm 2 Density-Aware Mutation with Heuristic Guidance

Require: Individual x, diversity score d, generation g, task-node rankings $\mathcal{R}$
Compute mutation budget $B \leftarrow f(g, d)$ (larger when diversity decreases)
for $b = 1$ to B **do**
  Sample a task index i
  Set x_i using heuristic-ranked nodes or a random fallback
end for
Apply a low-probability gene-level mutation pass
Optionally perform a small greedy local repair
return mutated individual x

4.4 Archive Guidance and Objective-Focused Final Candidate Pool

ADARE maintains a bounded archive of strong non-dominated or high-value candidates. During evolution, archive injection is rate-limited to guide search without collapsing diversity. At the end of the run, ADARE separates two roles: (i) the final non-dominated population for Pareto-quality metrics (HV, IGD, spacing, epsilon, coverage), and (ii) an objective-focused pool augmented with archive and targeted candidates for best-objective reporting (e.g., makespan or latency extremes). Algorithm 3 details this construction.

Algorithm 3 Archive-Guided Final Objective Pool Construction

Require: Final population P, archive A, targeted candidate set K
$P^{\text{obj}} \leftarrow P \cup A$
for all $k \in K$ **do**
  if k improves at least one tracked objective extreme **then**
    Insert k into P^{obj}
  end if
end for
return P for Pareto-quality metrics and P^{obj} for objective-best metrics

4.5 Complexity and Measured Control Overhead

The schedule evaluator dominates runtime. For population size P, generation count G, task count $|V|$, and average dependency-processing factor $\bar{d}$, evaluation cost is approximately

$$\mathcal{O}(G \cdot P \cdot |V| \cdot \bar{d}). \tag{7}$$

The contextual UCB controller adds constant-time operations per operator decision: context construction, four-operator UCB scoring, reward calculation on two parents and two children, and one exponential update. The context table has only 12×4 entries, and archive operations are bounded on a small structure. In this artifact, the schedule evaluator still dominates wall-clock time for realistic workflow sizes.

To verify this empirically, we instrumented controller selection/update, together with archive routines, on representative runs. The earlier global-controller

Table 2. Local Host Machine (from system diagnostics)

Item	Specification
System model	ASUS TUF Gaming A15 FA506NF_FA506NF (ASUSTeK COMPUTER INC.)
Host Operating System (OS)	Windows 11 Home 64-bit, Build 26200 (dxdiag timestamp: 2025-12-18)
Central Processing Unit (CPU)	AMD Ryzen 5 7535HS with Radeon Graphics, 12 logical CPUs, 3.300 GHz nominal
Random Access Memory (RAM)	16.000 GB installed (15.690 GB available in dxdiag)
Graphics Processing Unit(s) (GPU)	AMD Radeon(TM) Graphics + NVIDIA GeForce RTX 2050 (3.964 GB dedicated reported)
Display / mode	1920×1080; host reports 144 Hz internal panel and 75 Hz external display

version measured an upper-bound control ratio of **5.17% on average** (maximum **5.34%**) of total ADARE runtime. The revised contextual controller keeps the same asymptotic order and a fixed 12×4 table; the additional reward components are pairwise parent-child comparisons and do not call HV or IGD inside the loop. We therefore expect similar overhead, and the artifact now exports controller traces so this claim can be checked directly in reruns.

5 Experimental Protocol and Results

5.1 Artifact, Local Execution Context, and Hardware Configuration

All experiments were executed locally using the code in the current ADARE artifact (no cluster). We report the host details from system diagnostics because runtime differences at this scale are sensitive to workstation-level CPU and memory behavior.

The artifact contains the Python implementation, JSON workflow/resource inputs, configuration files, plotting code, and paired-run driver. Runnable entry points are grouped under the scripts directory, including the paired-run driver and ADARE-only runner. ADARE parameters are stored in the configuration directory; benchmark inputs are under the benchmark data directory; and controller traces are exported as per-benchmark trace and summary CSV files. This makes the adaptive-control decisions auditable rather than implicit in the final Pareto fronts.

Table 2 is operationally important because runtime gains in Python MOEAs depend on CPU generation and memory behavior. This became especially relevant when some longer runs were split into batches for execution stability. The local software stack used for the experiments is Python 3.13.1 with `numpy 2.2.5`, `matplotlib 3.10.8`, `deap 1.4`, `scipy 1.16.3`, and `pymoo 0.6.1.6`.

5.2 Benchmarks and Heterogeneous Resource Configuration

We evaluate three workflow instances commonly used in scientific workflow scheduling: `Montage_25`, `CyberShake_30`, and `Epigenomics_24`. These workflow families are available through public repositories used in the Pegasus and WfCommons ecosystems [6].

The heterogeneous resource model is defined in the experiment configuration. The tier template is aligned with the Edge-Fog Cloud reference setting proposed by Mohan and Kangasharju [2]. Table 3 reproduces the exact tier-level parameters used in the artifact.

Table 3. Resource Configuration Used in Experiments

Tier	Devices	Proc. rate	Cost	Idle P.	Work P.	Up/Down BW
Edge	5	1000	0.02	30	700	20480 / 20480
Fog	5	1300	0.48	30	700	10000 / 10000
Cloud	5	1600	0.96	1332	1648	100 / 10000

Devices: number of nodes; Proc. rate: processing rate in million instructions per second (MIPS); Cost: processing cost per second; Idle P./Work P.: power in watts; Up/Down BW: uplink/downlink bandwidth in megabits per second.

This table creates the optimization tension used throughout the study. In early debugging runs, it was easy to over-focus on cloud processing rate alone; the bandwidth asymmetry and cost/power gradients explain why that intuition is often wrong.

5.3 Fairness Protocol, Metrics, and Statistical Testing

ADARE and NSGA-III share the same evaluation function, generation budget, and initial population at each paired run. The artifact generates a shared initial population from the same seed before instantiating both algorithms, which enforces a fair comparison baseline.

We report **20 paired runs per benchmark**. In our runs on the workstation, benchmarks are executed sequentially (one workflow family at a time) to avoid runtime perturbations from concurrent processes. The statistical design is unchanged: paired 20-run comparisons.

Reported outputs include: (i) Pareto-quality metrics (HV, IGD, spacing, epsilon, coverage), (ii) runtime, and (iii) best objective values for makespan, latency, cost, and energy. For statistical validation, we use the paired Wilcoxon signed-rank test and Cliff's delta effect size [18,19].

5.4 Main Results: 20-Run Paired Comparison (ADARE vs NSGA-III)

Table 4 summarizes the main comparative outcomes (means over 20 paired runs). NSGA-III means are shown in parentheses for direct readability.

Table 4. 20-Run Summary (ADARE vs NSGA-III, means; NSGA-III in parentheses)

Benchmark	HV	IGD	Δ Mk. (%)	Δ Lat. (%)	Δt (%)
Montage_25	0.9493 (0.9232)	0.0600 (0.0624)	19.01	11.77	4.89
CyberShake_30	0.9645 (0.9303)	0.0633 (0.0718)	5.17	8.01	7.47
Epigenomics_24	1.0453 (1.0337)	0.0342 (0.0414)	1.18	4.34	4.13

Table 5. Statistical Evidence (20 Paired Runs): Wilcoxon p and Cliff's δ

Benchmark	Metric	Wilcoxon p	Cliff's δ
Montage_25	HV	1.05×10^{-4}	0.645 (large)
	Coverage	1.33×10^{-2}	0.440 (medium)
	Runtime	6.04×10^{-3}	0.725 (large)
CyberShake_30	HV	1.91×10^{-6}	0.980 (large)
	IGD	9.54×10^{-7}	0.940 (large)
	Runtime	1.97×10^{-4}	0.760 (large)
Epigenomics_24	HV	9.54×10^{-7}	0.960 (large)
	Spacing	9.54×10^{-7}	0.965 (large)
	Runtime	2.66×10^{-2}	0.325 (small)

ADARE improves HV on all three workflows and improves makespan/latency extremes in every case. In the final 20-run configuration, IGD also improves on all three workflows. This is useful because it means the stronger extremes and spread do not come at the expense of distance-to-reference quality in the reported protocol.

Table 5 confirms that the quality gains are not only visual. HV improvements are significant on all three workflows, and the selected runtime comparisons are also significant in this final protocol. Epigenomics nevertheless remains the benchmark where runtime gains are the smallest in effect size (small Cliff's δ), even when the direction is favorable.

In our runs across the three benchmarks, ADARE wins **18/18** quality comparisons (including runtime) and **15/15** core-quality comparisons (excluding runtime). Average gains are **26.80%** in core quality, **10.70%** in best-objective metrics, and **5.50%** in runtime.

5.5 Scalability Stress Test on 1000-Task Workflows

To address the scale concern raised by the reviews, we added a 20-repeat stress-test protocol on four 1000-task Pegasus-style workflows already available in the artifact: CyberShake_1000, Inspiral_1000, Montage_1000, and Sipht_1000. Each comparison uses the same evaluator, seeds, initial population policy, population size 80, 15 generations, and 10 parallel evaluator workers. This turns the earlier scalability check into a repeated large-instance protocol rather than a two-run smoke test.

Table 6. 1000-Task Confirmatory Stress Test (20 runs, core metrics: HV, IGD, spacing, epsilon, coverage)

Baseline	ADARE core wins	Mean core gain (%)	Mean HV gain (%)
NSGA-III	18/20	18.08	5.85
QL-NSGA-III	17/20	7.26	2.32
OVEA-style	20/20	49.74	48.11
QMOEA/D-AWA-style	17/20	29.44	40.69

Table 6 is important for interpretation. ADARE remains favorable on 72/80 core-quality comparisons at 1000 tasks, with an average core-quality gain of 26.13% across the four baselines. The difficult comparison is QL-NSGA-III, which is close on some metrics; nevertheless, the 20-run protocol moves the result from an ambiguous short-run tie to a positive repeated outcome. Against the two learning-based baselines highlighted by the reviews, ADARE improves mean core quality by +49.74% versus OVEA-style and +29.44% versus QMOEA/D-AWA-style.

5.6 Extended Baselines and 3000-Task WfCommons Workflows

We also added a broader comparison requested by the reviews. The extended runner evaluates ADARE against six baselines under the same evaluator, initial population, budget, and 10-worker setting: NSGA-III, NSGA-II, MOEA/D, QL-NSGA-III, OVEA-style, and QMOEA/D-AWA-style. OVEA-style couples reference-vector selection with Q-learning crossover choice. The QMOEA/D-AWA-style baseline couples decomposition, adaptive weights, and Q-learning neighborhood control. These are executable workflow adaptations; we do not import the original papers' numerical results because those results use different problem classes, objectives, encodings, and benchmarks.

The extended workflow set has three scales: (i) smaller local benchmarks, (ii) Pegasus 1000-task workflows already present in the artifact, and (iii) five WfCommons-generated 3000-task workflows (Montage, Epigenomics, Seismology, SoyKB, and SRA-search). Table 7 reports the broad screening suite, while Table 8 reports the 20-repeat confirmatory suite on the 1000-task and 3000-task workflows.

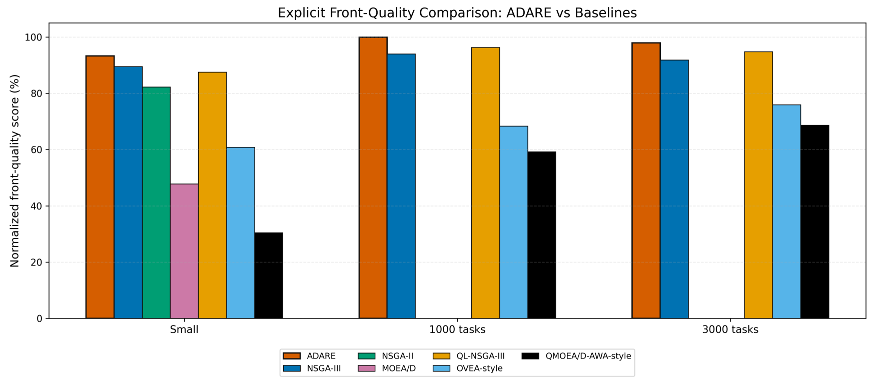

Fig. 3. Explicit normalized front-quality comparison between ADARE and all extended baselines.

Table 7. Extended Baseline Comparison (core metrics: HV, IGD, spacing, epsilon, coverage)

Scale	Baseline	ADARE core wins	Mean core gain (%)
Small	NSGA-III	13/25	21.73
Small	NSGA-II	17/25	135.87
Small	MOEA/D	20/25	77.59
Small	QL-NSGA-III	18/25	12.04
Small	OVEA-style	18/25	17.92
Small	QMOEA/D-AWA-style	21/25	152.31
1000 tasks	NSGA-III	12/20	3.06
1000 tasks	NSGA-II	14/20	40.87
1000 tasks	MOEA/D	16/20	23.57
1000 tasks	QL-NSGA-III	10/20	-3.05
1000 tasks	OVEA-style	17/20	45.78
1000 tasks	QMOEA/D-AWA-style	17/20	35.96
3000 tasks	NSGA-III	18/25	19.54
3000 tasks	NSGA-II	22/25	149.19
3000 tasks	MOEA/D	19/25	35.08
3000 tasks	QL-NSGA-III	16/25	0.95
3000 tasks	OVEA-style	22/25	139.04
3000 tasks	QMOEA/D-AWA-style	17/25	23.53

Table 8. Large-Scale 20-Run Confirmatory Protocol

Scale	Baseline	ADARE core wins	Mean core gain (%)	Mean HV gain (%)
1000 tasks	NSGA-III	18/20	18.08	5.85
1000 tasks	QL-NSGA-III	17/20	7.26	2.32
1000 tasks	OVEA-style	20/20	49.74	48.11
1000 tasks	QMOEA/D-AWA-style	17/20	29.44	40.69
3000 tasks	NSGA-III	23/25	19.26	7.92
3000 tasks	QL-NSGA-III	20/25	3.54	2.61
3000 tasks	OVEA-style	22/25	21.39	23.39
3000 tasks	QMOEA/D-AWA-style	21/25	17.92	22.11

The extended results are more conservative and more useful than the original NSGA-III-only story. ADARE remains favorable against NSGA-III at all three

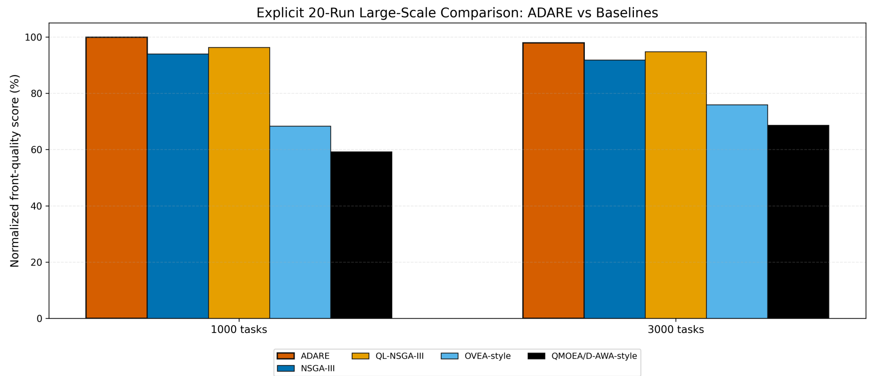

Fig. 4. Explicit normalized front-quality comparison between ADARE and adaptive baselines in the 20-repeat large-scale protocol.

scales, stays clearly ahead of NSGA-II and MOEA/D in the screening suite, and remains positive against the two added adaptive families in the 20-repeat large-scale protocol. OVEA-style is beaten strongly at 1000 and 3000 tasks, suggesting that reference-vector Q-learning operator adaptation alone is not enough for heterogeneous workflow schedules. QMOEA/D-AWA-style is more competitive but still trails ADARE by +29.44% mean core gain on 1000-task workflows and +17.92% on the 3000-task suite. QL-NSGA-III remains the strongest cautionary baseline: ADARE's gains are smaller against it, but still positive at both large scales. This shows that adaptive operator selection is a real competitive family; ADARE's contribution is the Pareto-contextual reward, diversity-aware mutation, archive guidance, and workflow-specific control design rather than the mere presence of learning.

5.7 Ablation Protocol (V1–V5) and Artifact-Ready Variant Definition

To isolate module effects in our runs, we keep the same paired-run fairness constraints and grow the ablation step by step. We start with **V1** (NSGA-III baseline), then add UCB crossover control in **V2**, density-aware mutation in **V3**, partial archive injection/final-pool guidance in **V4**, and full ADARE in **V5**.

Table 9. Numerical Ablation Results (V1–V5, 20 paired runs per benchmark)

Variant	UCB	Mut.	Arch.	Core (%)	Obj. (%)	Time (%)	Wins
V1	N	N	N	0.00	0.00	0.00	–
V2	Y	N	N	1.03	6.46	7.76	8/15
V3	Y	Y	N	1.95	7.29	8.63	11/15
V4	Y	Y	P	17.37	7.54	8.06	15/15
V5	Y	Y	Y	30.60	9.47	6.93	15/15

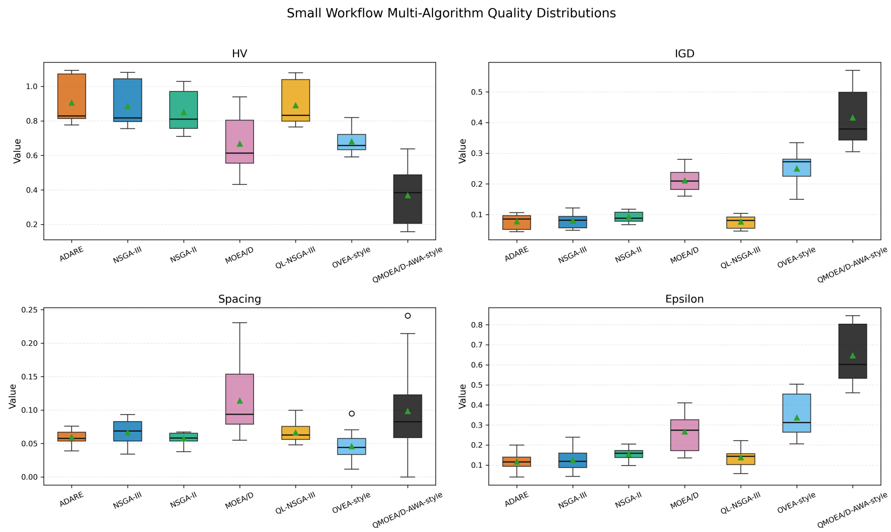

Fig. 5. Run-level distributions of HV, IGD, spacing, and epsilon on the rerun small-workflow suite with all baselines. ADARE is shown in vermilion; all algorithms use distinct colorblind-safe colors.

Numerically, V2 and V3 improve over V1 but remain less consistent (quality wins: 11/18 and 14/18), while V4 and V5 are both stronger (18/18), with V5 giving the best mean core gain. In Table 9, N/Y/P denote No/Yes/Partial. For a metric m, we compute $g_m(\%) = \frac{A_m - N_m}{|N_m|} \times 100$ for max metrics and $g_m(\%) = \frac{N_m - A_m}{|N_m|} \times 100$ for min metrics. Core (%) averages g_m over HV, IGD, spacing, epsilon, and coverage; Obj. (%) averages g_m over makespan_best, latency_best, cost_best, and energy_best; Time (%) is runtime gain; Wins counts positive core gains across the three benchmarks (out of 15). Positive values indicate ADARE gains.

5.8 Explicit Multi-Algorithm Visual Analysis

We replace the earlier two-algorithm Pareto and convergence snapshots with multi-algorithm figures from the extended runner. This avoids a misleading visual emphasis on only NSGA-III and makes the comparison against NSGA-II, MOEA/D, QL-NSGA-III, OVEA-style, and QMOEA/D-AWA-style directly visible.

Figures 5–7 show the revised visual story more transparently. ADARE is no longer only contrasted with NSGA-III: the same rerun includes classical MOEAs and learning-assisted baselines. The boxplots expose metric dispersion, the normalized score figure aggregates front quality, objective extremes, and runtime under a common scale, and the heatmap identifies where gains are strongest or weakest for each workflow-baseline pair.

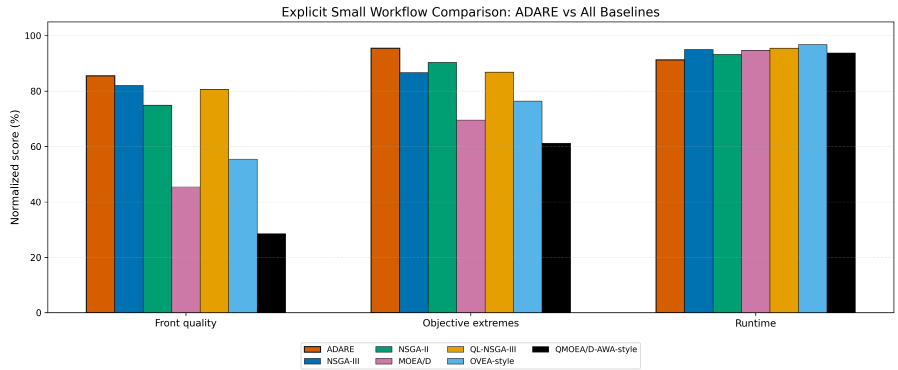

Fig. 6. Explicit normalized small-workflow comparison across front quality, objective extremes, and runtime. Each benchmark/run/metric is normalized so the best algorithm receives 100.

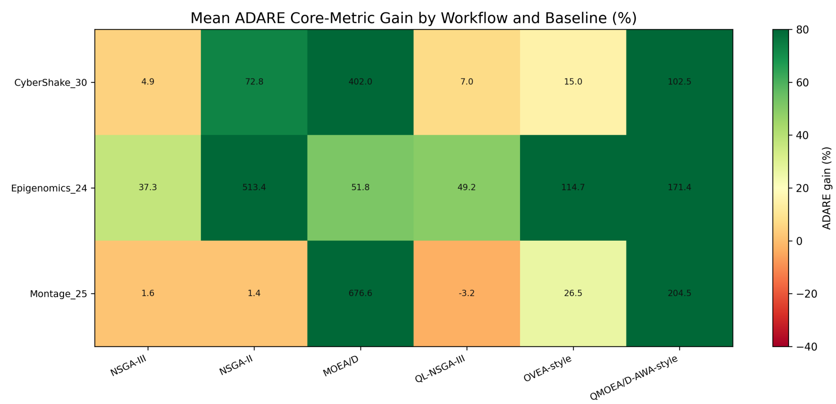

Fig. 7. Mean ADARE core-metric gain by workflow and baseline on the rerun small-workflow suite. Green cells indicate favorable ADARE margins; red cells identify competitive baselines or weaker metrics.

At this point, the practical question is less "does ADARE ever help?" and more "when does the adaptive layer help most, and where are its limits?" The discussion below addresses that question directly.

6 Discussion

6.1 Why Static NSGA-III Underperforms in Heterogeneous Environments

In our setting, the main weakness of a static NSGA-III pipeline is not the selection operator itself; NSGA-III's reference-point mechanism remains a strong many-objective baseline. The recurring issue is *operator control rigidity.* Fixed crossover and mutation schedules assume a reward landscape that stays roughly stable, but workflow scheduling across heterogeneous edge, fog, and cloud resources shows phase changes during search.

Early generations benefit from broad exploration to discover useful task-node assignments under precedence and bandwidth constraints. Later generations benefit more from operators that preserve partial structure and refine local trade-offs. A static schedule cannot represent this shift explicitly, while ADARE's controller adapts operator preference from observed rewards. The convergence plots were useful here because they exposed this phase dependence before the aggregate statistics made it obvious.

6.2 Justification of Experimental Choices

Population size and generation count. We use population size 100 and 70 generations in the main 20-run protocol to keep comparisons computationally affordable on a local workstation while preserving enough diversity for four-objective optimization. For 1000-task and 3000-task stress tests we use smaller budgets (population 80/60 and 15/8 generations, respectively) but keep 20 repetitions and 10 evaluator workers. This is a deliberate compromise between statistical replication and per-run search depth. Many-objective populations can become largely non-dominated early in the run, reducing selection pressure [22]; a future long-budget stress run (e.g., 500 generations) would test whether the observed advantage persists under substantially deeper convergence pressure.

Seeding and fairness. Seed control is not a cosmetic choice. In stochastic MOEAs, comparing unmatched runs can confound algorithmic differences with initialization variance. Our paired design uses the same initial population for ADARE and NSGA-III for every run, which makes Wilcoxon and Cliff's delta more informative.

Why 20 runs. The original artifact analysis used 6 runs. In our runs, that count exposed strong effects but remained fragile for smaller effect-size comparisons, especially runtime relative to HV/IGD/spacing gains. We moved the main protocol and the 1000/3000-task confirmatory stress tests to 20 runs to stabilize mean estimates and effect sizes. Table 5 shows that directions remain favorable while effect sizes still vary by benchmark and metric.

6.3 Sensitivity and Robustness Trends

Our observations suggest three sensitivity patterns.

1. **Run count sensitivity**: metrics with smaller effect sizes (most clearly runtime on Epigenomics in our final protocol) are sensitive to the number of repetitions and should not be interpreted from very small samples.

2. **Mutation/Archive sensitivity**: ADARE's strongest empirical gains are associated with spacing, coverage, and objective extremes, indicating that density-aware mutation and archive guidance dominate the observed improvements in difficult landscapes.
3. **Benchmark dependence**: ADARE delivers strong quality gains on all three workflows, but quality and runtime do not move with the same amplitude on every instance. Epigenomics shows a smaller runtime effect size than CyberShake or Montage despite clear quality improvements.

These patterns support the adaptive-control interpretation in our runs. Under the reported 20-run protocol, ADARE is superior on all tracked quality metrics, yet the *magnitude* of improvement still depends on benchmark structure. For that reason, we avoid benchmark-independent effect-size claims from smaller repetition counts, even when win/loss counts look unambiguous.

6.4 Reward Validity, Interpretability, and Scalability Limits

The revised controller directly addresses the reward-scale concern raised by the reviews: objective improvements are normalized by current population ranges, and the reward includes dominance and balanced-progress terms instead of relying on a fixed log-scalar shortcut. We still avoid per-mating HV computation because exact HV contribution is too expensive to use inside the inner loop for every crossover event. Since HV remains a strong unary quality indicator for Pareto-front assessment [23], the next artifact-level validation should correlate exported controller rewards with sampled HV-contribution rewards on a subset of events.

The implementation now exports controller traces containing generation, context, operator, reward, diversity, and stagnation. These traces make it possible to build the heatmap requested in the detailed review and to verify whether ADARE actually changes behavior across early, middle, late, low-diversity, and stagnation contexts.

The largest remaining limitation is external validity at scale. The added 1000-task and WfCommons 3000-task stress tests strengthen the scalability evidence with 20 repetitions, and the extended baseline suite now includes NSGA-II, MOEA/D, QL-NSGA-III, OVEA-style, and QMOEA/D-AWA-style comparisons. However, these large runs still use short generation budgets, and OVEA-style/QMOEA/D-AWA-style are executable workflow adaptations rather than exact imported artifacts. They therefore do not replace a broader large-scale study with longer budgets, more workflow domains, and exact public implementations of OVEA, QMOEA/D-AWA, or MRL-MOEA when such artifacts are available. We treat the current evidence as strong for controlled workflow scheduling and materially stronger for 1000–3000 task scale, not as a universal scalability claim.

7 Conclusion

We reframed ADARE as a Pareto-contextual online control layer for many-objective workflow scheduling rather than as a simple scheduler variant. This framing is technically useful because it explains why operator-level learning is central to the method, while the evolutionary backbone can remain largely unchanged.

Under a fairness-controlled protocol (same initialization, same budget, same evaluator) and a stronger statistical setup (20 paired runs per benchmark), ADARE improves overall many-objective search quality and runtime against NSGA-III on three heterogeneous workflow instances. In our runs, gains are strongest in HV, coverage, spacing, and objective extremes, and CyberShake shows the clearest combined benefit. Runtime also improves on all three benchmarks in the final protocol, although the Epigenomics runtime effect remains smaller than its quality effects. The 20-repeat 1000-task and 3000-task stress tests further show favorable scaling behavior, and the extended baseline suite confirms that ADARE remains competitive beyond a single NSGA-III comparison and beyond generic learning-based operator adaptation. Measured control overhead stays near 5% on average, preserving practical usability on a local workstation.

For the revised journal framing, the main takeaway is that workflow scheduling in heterogeneous distributed systems benefits from treating evolutionary operator choice as a *contextual online learning control problem*. Future work will extend this evaluation to larger workflow instances, add the 500-generation convergence stress test, validate the controller reward against sampled HV contribution, and replace the current OVEA-style and QMOEA/D-AWA-style adaptations with exact public implementations if the original artifacts become available.

References

1. Topcuoglu, H., Hariri, S., Wu, M.-Y.: Performance-effective and low-complexity task scheduling for heterogeneous computing. IEEE Transactions on Parallel and Distributed Systems **13**(3), 260–274 (2002)
2. Mohan, N., Kangasharju, J.: Edge-Fog Cloud: A Distributed Cloud for Internet of Things Computations. Presented at IEEE CIoT (2016)
3. Bonomi, F., Milito, R., Zhu, J., Addepalli, S.: Fog computing and its role in the Internet of Things. In: MCC Workshop on Mobile Cloud Computing, pp. 13–16 (2012)
4. Deb, K., Pratap, A., Agarwal, S., Meyarivan, T.: A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation **6**(2), 182–197 (2002)
5. Zhang, Q., Li, H.: MOEA/D: A multiobjective evolutionary algorithm based on decomposition. IEEE Transactions on Evolutionary Computation **11**(6), 712–731 (2007)
6. Coleman, T., Babuji, Y., Li, H., et al.: WfCommons: A framework for enabling scientific workflow research and development. Scientific Data **9**, 220 (2022).
7. Deb, K., Jain, H.: An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems

with box constraints. IEEE Transactions on Evolutionary Computation **18**(4), 577–601 (2014)
8. Auer, P., Cesa-Bianchi, N., Fischer, P.: Finite-time analysis of the multiarmed bandit problem. Machine Learning **47**, 235–256 (2002)
9. Da Costa, L., Fialho, A., Schoenauer, M., Sebag, M.: Adaptive operator selection with dynamic multi-armed bandits. In: GECCO, pp. 913–920 (2008)
10. Fialho, A., Da Costa, L., Schoenauer, M., Sebag, M.: Analyzing bandit-based adaptive operator selection mechanisms. Annals of Mathematics and Artificial Intelligence **60**(1–2), 25–64 (2010)
11. Besbes, O., Gur, Y., Zeevi, A.: Stochastic multi-armed-bandit problem with non-stationary rewards. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 27 (2014)
12. Jiao, K., Chen, J., Xin, B., Li, L.: OVEA: A reference vector based multiobjective evolutionary algorithm with Q-learning for operator adaptation. Evolutionary Computation **31**(3), 375–400 (2023)
13. Song, A., Wang, J., Li, K., Zhang, Q.: Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities. ACM Computing Surveys **56**(12), Article 311 (2024)
14. Wang, K., Wang, Y., Xie, X., Yu, S., Zhang, Q.: MRL-MOEA: Multi-role learning based many-objective evolutionary algorithm. Scientific Reports **15**, 25039 (2025)
15. Wang, Z., He, M., Chen, H., Hu, Y., Xia, Y.: A Q-learning-based evolutionary algorithm for solving the low-carbon multi-objective flexible job shop scheduling problem. Computers & Operations Research **185**, 107266 (2026)
16. Cui, W., Song, A., Wang, J., Zhang, Q.: DRLMMEA: Deep reinforcement learning-assisted many-objective evolutionary algorithm. Nature Communications **17**, 1084 (2026)
17. Mao, Y., You, C., Zhang, J., Huang, K., Letaief, K.B.: A survey on mobile edge computing: The communication perspective. IEEE Communications Surveys & Tutorials **19**(4), 2322–2358 (2017)
18. Wilcoxon, F.: Individual comparisons by ranking methods. Biometrics Bulletin **1**(6), 80–83 (1945)
19. Cliff, N.: Dominance statistics: Ordinal analyses to answer ordinal questions. Psychological Bulletin **114**(3), 494–509 (1993)
20. Karafotias, G., Hoogendoorn, M., Eiben, A.E.: Parameter control in evolutionary algorithms: Trends and challenges. IEEE Transactions on Evolutionary Computation **19**(2), 167–187 (2015)
21. Coello Coello, C.A., Lamont, G.B., Van Veldhuizen, D.A.: Evolutionary Algorithms for Solving Multi-Objective Problems, 2nd edn. Springer (2007)
22. Purshouse, R.C., Fleming, P.J.: On the evolutionary optimization of many conflicting objectives. IEEE Transactions on Evolutionary Computation **11**(6), 770–784 (2007)
23. Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C.M., Grunert da Fonseca, V.: Performance assessment of multiobjective optimizers: An analysis and review. IEEE Transactions on Evolutionary Computation **7**(2), 117–132 (2003)
24. Deelman, E., Singh, G., Su, M.-H., et al.: Pegasus: A framework for mapping complex scientific workflows onto distributed systems. Scientific Programming **13**(3), 219–237 (2005)